MANUAL TÉCNICO

MVP

PROYECTO INTERMODULAR 2025/2026

KultuX
donde la cultura
se encuentra contigo

Equipo KultuX Team:
CRISTO MACÍAS IZAGUIRRE
SANDRA MARÍA MOÑINO GARCÍA

Grupo: 2ºDAM - B
Módulo: 0492 · Proyecto Intermodular · 2025/2026
Docente tutor/a: MARÍA MERCEDES MARTÍNEZ FRAGOSO
IES ALBARREGAS

ENLACES:

Repositorio GitHub	https://github.com/KultuX
Trello	https://trello.com/b/llyc4vMl/pidam2b-kultuxteam-kultux
Figma	https://www.figma.com/design/lFKUtYHRuxWjA64pWLJIKE/KultuX?node-id=0-1&t=J3DjlOomKKaXRwlK-1

Fecha: 16/03/2026
Versión: v1

ÍNDICE

1. Descripción de lo implementado en esta evaluación **3**
Backend – API REST con Spring Boot 4
Base de Datos – PostgreSQL con Supabase 5
Frontend – Aplicación móvil con Flutter 6
Despliegue del sistema 6
2. Arquitectura del sistema **7**
Diagrama de arquitectura 7
Componentes de la arquitectura 8
3. Dependencias y versiones **11**
3.6 Versionado de dependencias 11
4. Estructura del repositorio. **12**
5. Endpoints o funcionalidades principales **13**
Microservicio de Actividades 13
Microservicio de Restaurantes 15
Microservicio de Alojamientos 16
Microservicio de Localidades **17**
Microservicio de Usuario y Empresa **18**
Base de datos **21**
Modelo de datos 21
Modelo Entidad-Relación 29
7 Pruebas. **30**
7.1 Tipos de pruebas realizadas 30
7.2 Funcionalidades verificadas 31
7.3 Funcionalidades pendientes o parcialmente implementadas 32
7.4 Cómo se ejecutan las pruebas 33
7.5 Evidencias 33
7.6 Resumen de estado actual 34
8. Problemas conocidos y limitaciones actuales **34**
9. Próximos pasos **35**

1. Descripción de lo implementado en esta evaluación

Durante esta evaluación se ha desarrollado el Producto Mínimo Viable (PMV) del proyecto, implementando una arquitectura basada en microservicios, una API REST, una base de datos relacional (Supabase) y una aplicación móvil encargada de interactuar con el usuario.

El objetivo principal ha sido construir la estructura básica del sistema que permita gestionar información relacionada con usuarios, actividades, alojamientos, restaurantes y localidades, así como permitir su consulta desde una aplicación móvil.

Para ello se han utilizado diferentes tecnologías y herramientas que permiten separar responsabilidades entre las distintas partes del sistema.

Backend – API REST con Spring Boot

El backend del sistema ha sido desarrollado utilizando Spring Boot, implementando una arquitectura por capas que separa claramente las responsabilidades de cada componente.

Esta arquitectura se divide principalmente en:

- Controladores (Controllers): gestionan las peticiones HTTP que llegan a la API.
- Servicios (Services): contienen la lógica de negocio de cada funcionalidad.
- Repositorios (Repositories): se encargan de la comunicación con la base de datos.

Además, el backend se ha estructurado siguiendo un enfoque de microservicios, donde cada servicio es responsable de un conjunto específico de datos y funcionalidades.

Los microservicios implementados son:

- Micro-Alojamiento: encargado de gestionar la información de alojamientos.
- Micro-Actividad: responsable de las actividades o eventos disponibles.
- Micro-Usuario: encargado de la gestión de usuarios del sistema.
- Micro-Restaurante: encargado de la información relacionada con restaurantes.
- Micro-Localidades: responsable de gestionar los datos de localidades.

Cada microservicio expone sus propios endpoints REST y contiene la lógica necesaria para interactuar con los datos que gestiona.

Para la gestión de conexiones con la base de datos se ha utilizado HikariCP, que permite optimizar el uso de conexiones y mejorar el rendimiento del sistema.

Base de Datos – PostgreSQL con Supabase

La base de datos del sistema ha sido implementada utilizando Supabase, que proporciona una base de datos PostgreSQL gestionada en la nube.

Se han creado diferentes tablas organizadas en esquemas, con el objetivo de separar los distintos dominios de datos del sistema.

Cada esquema agrupa las tablas relacionadas con una funcionalidad concreta y solo es accesible por el microservicio encargado de gestionar esos datos.

Esta organización permite:

- Mejor separación lógica de la información.
- Mayor seguridad en el acceso a los datos.
- Una estructura más clara y escalable.

Frontend – Aplicación móvil con Flutter

La interfaz de usuario del sistema ha sido desarrollada utilizando Flutter, framework multiplataforma que permite construir aplicaciones móviles modernas.

La aplicación móvil se comunica con la API REST para:

- Solicitar información al backend.
- Mostrar los datos en pantalla al usuario.
- Recoger información introducida por el usuario.

Enviar solicitudes al servidor tras realizar las validaciones necesarias.

El diseño de la interfaz se ha desarrollado teniendo en cuenta principios básicos de experiencia de usuario (UX) y diseño de interfaz (UI), facilitando la interacción con las distintas funcionalidades del sistema.

Despliegue del sistema

La API ha sido desplegada utilizando Render, un servicio de tipo PaaS (Platform as a Service) que permite alojar aplicaciones web en la nube.

Debido a las limitaciones de la versión gratuita de Render, el servicio se suspende automáticamente tras un periodo de inactividad. Para evitarlo, se ha configurado un CronJob que realiza una petición automática a la API cada 5 minutos, manteniendo el servicio activo y disponible.

2. Arquitectura del sistema

El sistema desarrollado sigue una arquitectura basada en microservicios, donde cada componente del sistema tiene una responsabilidad específica y se comunica con el resto mediante una API REST.

Esta arquitectura permite una mayor modularidad, escalabilidad y mantenimiento del sistema, ya que cada microservicio gestiona un dominio funcional concreto y accede únicamente a los datos que le corresponden.

La aplicación se divide en tres grandes capas:

- Front-End (Aplicación móvil)
- Back-End (API basada en microservicios)
- Base de datos
- Diagrama de arquitectura

Diagrama de arquitectura

Componentes de la arquitectura

Aplicación móvil (Front-End)

La interfaz de usuario ha sido desarrollada utilizando Flutter, framework multiplataforma que permite crear aplicaciones móviles a partir de una única base de código.

La aplicación es responsable de:

- Mostrar la información obtenida desde la API.
- Permitir al usuario interactuar con el sistema.
- Realizar peticiones HTTP a los distintos endpoints del backend.
- Procesar y representar los datos recibidos.

El cliente Flutter consume los servicios de la API mediante peticiones HTTP REST, recibiendo las respuestas en formato JSON.

API Backend (Spring Boot)

El backend del sistema ha sido desarrollado utilizando Spring Boot, implementando una arquitectura basada en microservicios.

Cada microservicio es responsable de gestionar un conjunto específico de datos y funcionalidades, permitiendo así separar claramente las responsabilidades dentro del sistema.

La API se encarga de:

- Gestionar la lógica de negocio.
- Procesar las peticiones del cliente.
- Acceder a la base de datos.
- Devolver los resultados en formato JSON.

Cada microservicio se estructura siguiendo una arquitectura por capas, compuesta por:

- Controller: gestiona las peticiones HTTP.
- Service: contiene la lógica de negocio.
- Repository: gestiona el acceso a la base de datos.

Microservicios del sistema

El sistema está compuesto por los siguientes microservicios:

Micro-Usuario

Responsable de la gestión de los usuarios del sistema.

Funciones principales:
- Registro de usuarios
- Autenticación
- Gestión de perfiles
- Consulta de información de usuario

Micro-Actividad

Gestiona las actividades y eventos disponibles en la aplicación.

Funciones principales:
- Consulta de actividades
- Obtención de actividades destacadas
- Consulta del detalle de una actividad

Micro-Restaurante

Gestiona la información relacionada con restaurantes.

Funciones principales:
- Consulta de restaurantes
- Obtención de restaurantes destacados
- Consulta de detalle de restaurante

Micro-Alojamiento

Encargado de la gestión de alojamientos disponibles.

Funciones principales:

- Consulta de alojamientos
- Obtención de alojamientos destacados
- Consulta del detalle de alojamiento

Micro-Localidades

Gestiona las localidades disponibles en el sistema.

Funciones principales:

- Consulta de localidades
- Relación de localidades con actividades, restaurantes y alojamientos.

Base de datos

La base de datos del sistema se encuentra alojada en Supabase, utilizando el motor PostgreSQL.

La información se organiza en diferentes esquemas, cada uno relacionado con un dominio funcional concreto.

Cada microservicio accede únicamente a los esquemas y tablas correspondientes a su dominio, lo que permite mantener una mayor separación de responsabilidades y mejorar la seguridad en el acceso a los datos.

Despliegue

La API se encuentra desplegada en Render, un servicio PaaS que permite alojar aplicaciones backend en la nube.

Debido a las limitaciones de la versión gratuita del servicio, se ha configurado un CronJob que realiza una petición periódica a la API cada cinco minutos para evitar que el servicio se suspenda por inactividad.

3. Dependencias y versiones

Componente	Tecnología	Función
Backend	Java + Spring Boot	Desarrollo de la API y microservicios
Persistencia	Spring Data JPA	Acceso a base de datos
Pool de conexiones	HikariCP	Gestión eficiente de conexiones
Frontend	Flutter + Dart	Desarrollo de la aplicación móvil
Base de datos	PostgreSQL	Almacenamiento relacional de datos
Plataforma BD	Supabase	Gestión cloud de la base de datos
Despliegue	Render	Hosting del backend
Automatización	CronJob	Mantener activo el servicio desplegado
Control de versiones	Git / GitHub	Gestión del código fuente
Gestión del proyecto	Trello	Organización de tareas

3.6 Versionado de dependencias

Las versiones concretas de cada dependencia pueden consultarse en los archivos de configuración de cada entorno de desarrollo:

En el backend, a través del archivo de construcción pom.xml.

En el frontend, a través del archivo pubspec.yaml de Flutter.

En la base de datos, mediante la configuración del entorno Supabase y los scripts SQL utilizados en la creación de esquemas y tablas.

4. Estructura del repositorio.

En GitHub, no tenemos un único repositorio, sino que hemos creado uno para cada microservicio o para el front.

La estructura es la siguiente:

KultuX

- micro-usuario
- micro-restaurante
- micro-alojamiento
- micro-actividad
- micro-localidades
- micro-interaccion (v2)
- api-gateway (v2)
- kultux-front
- kultux-docs

En esta primera versión, no están todos en uso, puesto que son para funcionalidades futuras. Se indica en la parte superior cuales se espera que funcionen en la segunda versión del proyecto.

La documentación técnica se encontrará en kultux-docs.

5. Endpoints o funcionalidades principales

El sistema expone diferentes endpoints REST a través de los microservicios desarrollados con Spring Boot. Estos endpoints permiten a la aplicación móvil realizar consultas y obtener información sobre las distintas entidades del sistema, como actividades, restaurantes, alojamientos, usuarios y localidades.

Las peticiones se realizan mediante el protocolo HTTP, y las respuestas se devuelven en formato JSON.

A continuación se describen los principales endpoints implementados en los microservicios.

Microservicio de Actividades

Este microservicio gestiona la información relacionada con las actividades y eventos disponibles en la aplicación.

Obtener actividades destacadas

Permite obtener un listado de actividades destacadas que se muestran en la pantalla principal de la aplicación.

Endpoint

https://micro-actividad.onrender.com/api/v1/actividades/destacados

Respuesta

[
{
"id": 1,
"titulo": "Concierto Rock",
"categoriaActividad": "MUSICA",
"fechaInicio": "2026-03-20",
"localidad": "Mérida",
"imagenPrincipal":
"https://tvpvkxenflxqqjyahzus.supabase.co/storage/v1/object/sign/imagenes_usuarios/istockphoto-1330424071-612x612.jpg?token=eyJraWQiOiJzdG9yYWdlLXVybC1zaWduaW5nLWtleV83M2YyNmE1Yi1lOGRhLTRkZDYtOGQwNS1kM2M3ZmM4ZWI0

MWUiLCJhbGciOiJIUzI1NiJ9.eyJ1cmwiOiJpbWFnZW5lc191c3Vhcmlvcy9pc3RvY2twaG90by0xMzMwNDI0MDcxLTYxMng2MTIuanBnIiwiaWF0IjoxNzczNjIxOTE1LCJleHAiOjE4MDUxNTc5MTV9.SBypPumhDOFe4RXNZDHoHL53TsBBXXbhDqcELSh3vak"
}
]

Obtener detalle de una actividad

Permite consultar toda la información detallada de una actividad concreta.

Endpoint

https://micro-actividad.onrender.com/api/v1/actividades/{id}

https://micro-actividad.onrender.com/api/v1/actividades/2

Respuesta

{
"id": 2,
"titulo": "Obra de Teatro Infantil",
"localidad": "Badajoz",
"descripcion": "Espectáculo para niños",
"telefonoEmpresa": "924987654",
"correoEmpresa": "contacto@culturaviva.es",
"categoriaActividad": "TEATRO",
"fechaInicio": "2026-03-22",
"fechaFin": "2026-03-22",
"imagenPrincipal":
"https://tvpvkxenflxqqjyahzus.supabase.co/storage/v1/object/sign/imagenes_usuarios/05-Andereno-La-Maestra-Txelu-Angoitia-scaled.jpg?token=eyJraWQiOiJzdG9yYWdlLXVybC1zaWduaW5nLWtleV83M2YyNmE1Yi1lOGRhLTRkZDYtOGQwNS1kM2M3ZmM4ZWI0MWUiLCJhbGciOiJIUzI1NiJ9.eyJ1cmwiOiJpbWFnZW5lc191c3Vhcmlvcy8wNS1BbmRlcmVuby1MYS1NYWVzdHJhLVR4ZWx1LUFuZ29pdGlhLXNjYWxlZC5qcGciLCJpYXQiOjE3NzM2MjI1NTgsImV4cCI6MTgwNTE1ODU1OH0.RVDdetrSv01jTu5LS-2AwPrVNcLbKwklqYOFe2yPmII"
}

Microservicio de Restaurantes

Endpoint

Gestiona la información relacionada con los restaurantes disponibles en la plataforma.

Obtener restaurantes destacados

Devuelve un listado de restaurantes destacados que aparecen en la sección de establecimientos.

Endpoint

https://micro-restaurante-n4bv.onrender.com/api/v1/restaurantes/destacados

Respuesta

[
{
"id": 1,
"nombre": "A de Arco",
"categoriaRestaurante": "ALTA_COCINA",
"imagenPrincipal":
"https://tvpvkxenflxqqjyahzus.supabase.co/storage/v1/object/sign/imagenes_restaurantes/adearco.jpg?token=eyJraWQiOiJzdG9yYWdlLXVybC1zaWduaW5nLWtleV83M2YyNmE1Yi1lOGRhLTRkZDYtOGQwNS1kM2M3ZmM4ZWI0MWUiLCJhbGciOiJIUzI1NiJ9.eyJ1cmwiOiJpbWFnZW5lc19yZXN0YXVyYW50ZXMvYWRlYXJjby5qcGciLCJpYXQiOjE3NzM2MjM0ODAsImV4cCI6MTgwNTE1OTQ4MH0.Vh0WivLxU--vZgekupTmKfAERyjg4Oq-VM4IdgdxjG0"
}
]

Obtener detalle de un restaurante

Permite consultar la información completa de un restaurante específico.

Endpoint

https://micro-restaurante-n4bv.onrender.com/api/v1/restaurantes/{id}
https://micro-restaurante-n4bv.onrender.com/api/v1/restaurantes/2

Respuesta

{
"id": 2,
"nombre": "Sybarit Gastroshop",
"localidad": "Mérida",
"descripcion": "Bar gastronómico moderno con tapas creativas",
"telefonoEmpresa": "924987654",
"correoCorporativo": "contacto@culturaviva.es",
"categoriaRestaurante": "TAPAS",
"horarioApertura": "13:00-16:00 / 20:00-23:30",
"imagenPrincipal":
"https://tvpvkxenflxqqjyahzus.supabase.co/storage/v1/object/sign/imagenes_restaurantes/sybarit-comedor.jpg?token=eyJraWQiOiJzdG9yYWdlLXVybC1zaWduaW5nLWtleV83M2YyNmE1Yi1lOGRhLTRkZDYtOGQwNS1kM2M3ZmM4ZWI0MWUiLCJhbGciOiJIUzI1NiJ9.eyJ1cmwiOiJpbWFnZW5lc19yZXN0YXVyYW50ZXMvc3liYXJpdC1jb21lZG9yLmpwZyIsImlhdCI6MTc3MzYyMzUwMywiZXhwIjoxODA1MTU5NTAzfQ.796t8E7ZVOM0WXXLb4ZG1Q7uOZUXmCY-3tKM8mdq0zY"
}

Microservicio de Alojamientos

Gestiona los alojamientos disponibles dentro de la aplicación.

Obtener alojamientos destacados

Devuelve los alojamientos destacados que se muestran en la sección principal de establecimientos.

Endpoint

https://micro-alojamiento.onrender.com/api/v1/alojamientos/destacados

Respuesta

[
{
"id": 2,
"nombre": "Hotel Ilunion Mérida",
"categoriaAlojamiento": "HOTEL",
"imagenPrincipal":
"https://tvpvkxenflxqqjyahzus.supabase.co/storage/v1/object/sign/imagenes_alojamiento/ilunion-merida-palace.jpg?token=eyJraWQiOiJzdG9yYWdlLXVybC1zaWduaW5nLWtleV83M2YyNmE1Yi1lOGRhLTRkZDYtOGQwNS1kM2M3ZmM4ZWI0MWUiLCJhbGciOiJIUzI1NiJ9.eyJ1cmwiOiJpbWFnZW5lc19hbG9qYW1pZW50by9pbHVuaW9u

LW1lcmlkYS1wYWxhY2UuanBnIiwiaWF0IjoxNzczNjIzMDk4LCJleHAiOjE4MDUxNTkwOTh9.3_xmLW-4IwbPycKTZCsKyfXIK3xmvJ-aWPiUfo3Jf2Y"
}
]

Obtener detalle de un alojamiento
Permite consultar toda la información detallada de un alojamiento concreto.

Endpoint
https://micro-alojamiento.onrender.com/api/v1/alojamientos/{id}
https://micro-alojamiento.onrender.com/api/v1/alojamientos/2

Respuesta

{
"id": 2,
"nombre": "Hotel Ilunion Mérida",
"localidad": "Mérida",
"descripcion": "Hotel céntrico con encanto cerca del Teatro Romano",
"telefonoEmpresa": "924123456",
"correoCorporativo": "info@eventosmerida.es",
"categoriaAlojamiento": "HOTEL",
"imagenPrincipal":
"https://tvpvkxenflxqqjyahzus.supabase.co/storage/v1/object/sign/imagenes_alojamiento/ilunion-merida-palace.jpg?token=eyJraWQiOiJzdG9yYWdlLXVybC1zaWduaW5nLWtleV83M2YyNmE1Yi1lOGRhLTRkZDYtOGQwNS1kM2M3ZmM4ZWI0MWUiLCJhbGciOiJIUzI1NiJ9.eyJ1cmwiOiJpbWFnZW5lc19hbG9qYW1pZW50by9pbHVuaW9uLW1lcmlkYS1wYWxhY2UuanBnIiwiaWF0IjoxNzczNjIzMDk4LCJleHAiOjE4MDUxNTkwOTh9.3_xmLW-4IwbPycKTZCsKyfXIK3xmvJ-aWPiUfo3Jf2Y"
}

Microservicio de Localidades

Este microservicio permite gestionar la información relacionada con las localidades disponibles en el sistema.

Obtener listado de localidades
Devuelve todas las localidades registradas en el sistema.

Endpoint

https://micro-localidades.onrender.com/api/v1/localidades/nombres

Respuesta:


```
[
  {
    "ine": 10001,
    "nombre": "Abadía"
  }
]
```


Obtener localidad por id
Encuentra una localidad por su id

Endpoint
https://micro-localidades.onrender.com/api/v1/localidades/6083

Respuesta


```
{
  "ine": 6083,
  "nombre": "Mérida"
}
```


Microservicio de Usuario y Empresa

Gestiona la información relacionada con los usuarios de la aplicación.

Consultar empresa
Consulta la empresa que ha creado la actividad

Endpoint

https://micro-usuario.onrender.com/api/empresas/consultar?id=5

Respuesta


```
{
  "correoCorporativo": "info@eventosmerida.es",
  "telefonoEmpresa": "924123456"
}
```

Login usuario (Post)
Permite acceder al usuario tras validar que las credenciales son correctas

Endpoint
https://micro-usuario.onrender.com/api/usuarios/login

Body:
{
"email": "mercedes@gmail.com",
"password": "mercedes"
}

Respuesta
{
"id": 10,
"nombre": "Mercedes",
"email": "mercedes@gmail.com",
"localidad": 100037,
"fechaNacimiento": "1975-06-14",
"imagenPerfil":
"'https://tvpvkxenflxqqjyahzus.supabase.co/storage/v1/object/sign/imagenes_usuarios/perfil_bbdd.png?token=eyJraWQiOiJzdG9yYWdlLXVybC1zaWduaW5nLWtleV83M2YyNmE1Yi1lOGRhLTRkZDYtOGQwNS1kM2M3ZmM4ZWI0MWUiLCJhbGciOiJIUzI1NiJ9.eyJ1cmwiOiJpbWFnZW5lc191c3VhcmlvcY9wZXJmaWxfYmJkZC5wbmciLCJpYXQiOjE3NzM0ODA2MzAsImV4cCI6MTkzMTE2MDYzMH0.kB0GaNdQU8L9RFM1Ilf2gQX-B4ytQ9e_816aJdWR6rQ'::text"
}

Crear un nuevo usuario (Post)
Método para registrar a un nuevo usuario en la base de datos.

Endpoint
https://micro-usuario.onrender.com/api/usuarios/crear

Body:

{
"nombre": "Cristo",
"apellidos": "Macías Izaguirre",
"email": "cristo@gmail.com",
"password": "cristomacias",
"localidad": 6083,
"fechaNacimiento": "1995-04-18"
}

Respuesta

{
"email": "cristo@gmail.com"
}

Base de datos

Modelo de datos

Modelo de datos de KultuX. En él incluimos tablas, y tipos de datos.

USUARIO - Tabla que almacena los datos de los usuarios.

Nombre campo	Tipo de dato	LON.	DEC.
id_usuario	numérico	8	0
nombre	alfanumérico	20	
apellidos	alfanumérico	30	
correo_electronico	alfanumérico	320	
password	alfanumérico	30	
localidad	alfanumérico	80	
fecha_nacimiento	fecha	10	
rol	alfanumérico	enum	
imagen_perfil	alfanumérico	url	
fecha_registro	fecha y hora		

TABLA ACTIVIDAD

Nombre campo	Tipo de dato	LON.	DEC.
id_actividad	numérico	8	0
id_empresa	numérico	8	0
titulo	alfanumérico	50	
direccion	alfanumérico	80	
fecha_inicio	fecha y hora		
fecha_fin	fecha y hora		
precio	numérico	4	2
descripcion	alfanumérico	text	
total_asistentes	numérico	10	0
total_guardados	numérico	10	0
categoría	alfanumérico	enum	
estado	alfanumérico	enum	
asistencia_maxima	numérico	10	0

TABLA ALOJAMIENTO

Nombre campo	Tipo de dato	LON.	DEC.
id_alojamiento	numérico	8	0
id_empresa	numérico	8	0
categoria_alojamiento	alfanumérico	enum	
descripcion_alojamiento	alfanumérico	text	
total_guardados	numérico	8	0
direccion_alojamiento	alfanumérico	200	
telefono_alojamiento	alfanumérico	13	
nombre_alojamiento	alfanumérico	80	
localidad_alojamiento	alfanumérico	50	
provincia_alojamiento	alfanumérico	50	
web_alojamiento	alfanumérico	url	
web_reserva_alojamiento	alfanumérico	url	

TABLA RESTAURANTE

Nombre campo	Tipo de dato	LON.	DEC.
id_restaurante	numérico	8	0
id_empresa	numérico	8	0
categoria_restaurante	alfanumérico	enum	
descripcion_restaurante	alfanumérico	text	
total_guardados	numérico	8	0
direccion_restaurante	alfanumérico	200	
telefono_restaurante	alfanumérico	13	
nombre_restaurante	alfanumérico	80	
localidad_restaurante	alfanumérico	50	
provincia_restaurante	alfanumérico	50	
web_restaurante	alfanumérico	url	
web_reserva_restaurante	alfanumérico	url	

TABLA EMPRESA

Nombre campo	Tipo de dato	LON.	DEC.
id_empresa	numérico	8	0
id_usuario	numérico	8	0
nombre	alfanumérico	20	
CIF	alfanumérico	10	
email_corporativo	alfanumérico	250	
telefono_empresa	alfanumérico	12	
documento	documento		
dni_representante	numérico	8	0
logotipo	imagen		

TABLA COMENTARIO

Nombre campo	Tipo de dato	LON.	DEC.
id_comentario	numérico	8	0
id_actividad	numérico	8	0
id_usuario	numérico	8	0
comentario	texto		
fecha	fecha		
id_padre	numérico	8	0

TABLA ASISTENTE_ACTIVIDAD

Nombre campo	Tipo de dato	LON.	DEC.
id_actividad	numérico	8	0
id_usuario	numérico	8	0
participantes	numérico	10	0

TABLA GUARDADOS

Nombre campo	Tipo de dato	LON.	DEC.
id_actividad	numérico	8	0
id_usuario	numérico	8	0

TABLA IMAGEN_ACTIVIDAD

Nombre campo	Tipo de dato	LON.	DEC.
id_imagen	numérico	8	0
id_actividad	numérico	8	0
orden	numérico	8	0
es_portada	booleano		
url	texto		

TABLA IMAGEN_ALOJAMIENTO

Nombre campo	Tipo de dato	LON.	DEC.
id_imagen	numérico	8	0
id_alojamiento	numérico	8	0
orden	numérico	8	0
es_portada	booleano		
url	texto		

TABLA IMAGEN_RESTAURANTE

Nombre campo	Tipo de dato	LON.	DEC.
id_imagen	numérico	8	0
id_restaurante	numérico	8	0
orden	numérico	8	0
es_portada	booleano		
url	texto		

TABLA AUDITORIA_ACTIVIDAD

Nombre campo	Tipo de dato	LON.	DEC.
id_auditoria_actividad	numérico	8	0
id_actividad	numérico	8	0
id_empresa	numérico	8	0
id_usuario	numérico	8	0
accion	alfanumérico	20	
antes_cambios	json		
despues_cambios	json		
fecha_accion	fecha y hora		
descripcion	texto		

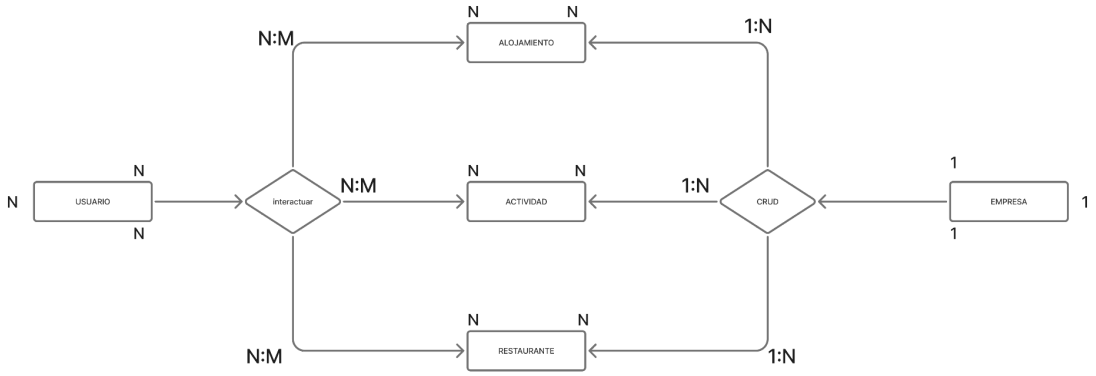
TABLA AUDITORIA_RESTAURANTE

Nombre campo	Tipo de dato	LON.	DEC.
id_auditoria_restaurante	numérico	8	0
id_restaurante	numérico	8	0
id_empresa	numérico	8	0
id_usuario	numérico	8	0
accion	alfanumérico	20	
antes_cambios	json		
despues_cambios	json		
fecha_accion	fecha y hora		
descripcion	texto		

TABLA AUDITORIA_ALOJAMIENTO

Nombre campo	Tipo de dato	LON.	DEC.
id_auditoria_alojamiento	numérico	8	0
id_alojamiento	numérico	8	0
id_empresa	numérico	8	0
id_usuario	numérico	8	0
accion	alfanumérico	20	
antes_cambios	json		
despues_cambios	json		
fecha_accion	fecha y hora		
descripcion	texto		

Modelo Entidad-Relación



7 Pruebas

Durante esta evaluación se han realizado pruebas funcionales manuales sobre las principales funcionalidades implementadas en el PMV, con el objetivo de comprobar el correcto funcionamiento de la aplicación, la comunicación entre el frontend y la API, y la recuperación de datos desde la base de datos.

Las pruebas se han centrado principalmente en validar los flujos principales de uso disponibles actualmente en la aplicación.

7.1 Tipos de pruebas realizadas

En el estado actual del proyecto se han realizado principalmente:

Pruebas funcionales manuales, verificando el comportamiento de las pantallas y flujos principales de la aplicación.

- Pruebas de integración básicas, comprobando la comunicación entre la aplicación Flutter, la API desarrollada con Spring Boot y la base de datos alojada en Supabase.

- Pruebas de navegación, asegurando que el usuario puede moverse entre las distintas pantallas disponibles según su estado de autenticación.

- Por el momento, no se han incorporado pruebas automatizadas unitarias o de interfaz, ya que el trabajo de esta evaluación se ha centrado en la implementación funcional del PMV.

7.2 Funcionalidades verificadas

Acceso y autenticación

Se ha comprobado el funcionamiento de los siguientes casos:

- Inicio de sesión de usuario mediante email y contraseña.
- Registro de nuevos usuarios enviando los datos requeridos al microservicio de usuario.
- Acceso como invitado, permitiendo utilizar la aplicación sin autenticación completa.

Estas pruebas han permitido validar tanto el envío de datos desde Flutter como el procesamiento de las peticiones en la API.

Carga de actividades en pantalla principal

Se ha verificado que, al acceder a la aplicación, se recuperan correctamente las actividades destacadas desde el backend y se muestran en la pantalla principal.

También se ha comprobado que el usuario puede:

- visualizar el listado de actividades disponibles
- seleccionar una actividad concreta
- acceder al detalle de la actividad

Esto confirma el correcto funcionamiento del flujo:

Flutter → API REST → Base de datos → Respuesta JSON → Renderizado en pantalla

Módulo de establecimientos

En la sección de establecimientos se han realizado pruebas sobre:

- carga de restaurantes destacados
- carga de alojamientos destacados

- acceso al detalle de un restaurante destacado
- acceso al detalle de un alojamiento destacado
- acceso al listado completo mediante la opción “Ver más”
- selección de cualquier restaurante del listado completo y consulta de su detalle
- selección de cualquier alojamiento del listado completo y consulta de su detalle

Estas pruebas han permitido validar tanto la visualización resumida como el acceso a listados completos y a vistas de detalle.

Control de acceso al perfil

Se ha comprobado que la pantalla de perfil solo está disponible cuando el usuario ha iniciado sesión.

En caso de que el usuario se encuentre en modo invitado y trate de acceder a esta sección:

- la aplicación bloquea el acceso
- se redirige de nuevo a la pantalla inicial con el componente de login
- se muestra un aviso informativo en la parte inferior de la interfaz

Esto permite validar la lógica de control de acceso implementada en el frontend.

7.3 Funcionalidades pendientes o parcialmente implementadas

Durante las pruebas también se ha identificado el estado actual de los módulos que todavía no están finalizados:

- Mapa: la pantalla está prevista en la navegación, pero su funcionalidad aún no ha sido implementada.

- Búsqueda y filtros: la interfaz muestra opciones de filtrado, pero la lógica de búsqueda y filtrado todavía no está conectada al backend.

Estas secciones forman parte del desarrollo pendiente para siguientes iteraciones del proyecto.

7.4 Cómo se ejecutan las pruebas

Las pruebas realizadas se ejecutan poniendo en marcha los distintos componentes del sistema y validando manualmente los flujos desde la aplicación móvil.

El proceso general de prueba es el siguiente:

1. Ejecutar o desplegar los microservicios y comprobar su disponibilidad.
2. Verificar la conexión con la base de datos en Supabase.
3. Ejecutar la aplicación Flutter.
4. Navegar por las distintas pantallas.
5. Comprobar la carga de datos, la navegación y el acceso a detalles.
6. Validar el comportamiento según el estado del usuario: autenticado o invitado.

Adicionalmente, algunos endpoints del backend pueden comprobarse mediante herramientas como Postman, enviando peticiones HTTP y verificando las respuestas JSON recibidas.

7.5 Pruebas para el usuario

Prueba en entorno de desarrollo (Obligatorio)

Para que el usuario pueda probar la versión inicial de KultuX, necesita tener instalado en su ordenador:

- Android Studio: Versión Hedgehog (2023.1.1) o superior
- Android SDK: API 33 o superior (Recomendado API 34)
- Flutter: 3.38.5
- Dart: 3.10.4
- Plugins de Android Studio:
  - Flutter
  - Dart
- Emulador recomendado: Pixel 8 (Android 34)

Una vez esté configurado el entorno de desarrollo, debe traer los datos del repositorio KultuX-front y ejecutarlo. Con este ya estaría funcional ya que los diferentes microservicios están desplegados en render.

Prueba APK en móvil
Una vez configurado el entorno, debemos escribir en consola el comando:

flutter build apk --release

Esto creará el APK que debemos instalar en el dispositivo móvil Android, el cual debe tener conexión a internet. Con esto funcionaría la primera versión de la aplicación.

7.6 Evidencias

Las evidencias más representativas para la defensa del proyecto son:

- login correcto
- acceso como invitado
- carga de actividades en inicio

- consulta de detalle de actividad
- acceso a establecimientos destacados
- acceso al listado completo de restaurantes y alojamientos
- consulta de detalle de establecimiento
- restricción de acceso al perfil en modo invitado

7.7 Resumen de estado actual

En la fase actual del proyecto, el PMV presenta un grado de funcionalidad suficiente para demostrar el flujo principal de uso de la aplicación. Se encuentran operativas las funciones de autenticación básica, consulta de actividades, consulta de establecimientos y visualización de detalles.

No obstante, algunos módulos como el mapa y la búsqueda avanzada continúan en desarrollo y deberán completarse en siguientes iteraciones antes de la defensa final.

8. Problemas conocidos y limitaciones actuales

Aunque el sistema es funcional en su flujo principal, actualmente existen algunas limitaciones propias del estado de desarrollo del Producto Mínimo Viable (PMV).

En primer lugar, el módulo de mapa todavía no se encuentra implementado. La pantalla está integrada en la navegación de la aplicación, pero la funcionalidad de visualización de ubicaciones y marcadores aún no ha sido desarrollada.

Por otro lado, la funcionalidad de búsqueda y filtrado se encuentra parcialmente implementada. La interfaz muestra las opciones de filtros, pero la

lógica necesaria para realizar las búsquedas en el backend todavía no está conectada.

Además, el backend está desplegado en Render utilizando el plan gratuito, lo que puede provocar que los servicios entren en estado de suspensión tras periodos de inactividad. Para evitarlo se ha configurado una petición periódica que mantiene activo el servicio.

Estas limitaciones forman parte del estado actual del PMV y están previstas para su mejora en las siguientes fases de desarrollo del proyecto.

9. Próximos pasos

De cara a la defensa final del proyecto y a la evolución futura del sistema, se prevé continuar ampliando y mejorando las funcionalidades actuales con el objetivo de convertir KultuX en una plataforma completa para la exploración cultural, turística y gastronómica de Extremadura.

En primer lugar, se completará la implementación del módulo de mapa, permitiendo mostrar actividades, restaurantes y alojamientos geolocalizados. Esta funcionalidad permitirá al usuario visualizar los diferentes puntos de interés directamente sobre el mapa y facilitará la exploración de eventos y establecimientos cercanos a su ubicación.

También se finalizará el sistema de búsqueda y filtrado, conectando la interfaz ya desarrollada con los microservicios del backend. Esto permitirá realizar búsquedas más precisas mediante filtros por criterios como tipo de actividad, localidad, fecha o categoría de establecimiento, mejorando significativamente la experiencia de usuario a la hora de descubrir contenido dentro de la aplicación.

Otro aspecto importante será la mejora del sistema de interacción del usuario, ampliando las funcionalidades disponibles en la aplicación. Entre las posibles mejoras se contemplan características como marcar actividades favoritas, confirmar asistencia a eventos, guardar establecimientos de interés o permitir valoraciones y comentarios por parte de los usuarios.

Asimismo, se continuará trabajando en la optimización de la interfaz y la experiencia de usuario, refinando la navegación entre pantallas, mejorando la

gestión de errores y ampliando las validaciones de los formularios para ofrecer una interacción más clara y robusta.

Desde el punto de vista técnico, también se prevé mejorar la arquitectura del sistema, incorporando nuevas funcionalidades a los microservicios existentes y ampliando las capacidades del API Gateway para gestionar de forma más eficiente las peticiones de la aplicación.

A largo plazo, el objetivo del proyecto es evolucionar KultuX hacia una plataforma capaz de centralizar la información cultural, turística y gastronómica de la región, facilitando tanto a residentes como a visitantes el descubrimiento de actividades, eventos y lugares de interés, convirtiéndose en una herramienta útil para dinamizar la oferta cultural y turística del territorio.